

Understanding Backend Development

Explore the essential concepts behind web application backends, from server architecture to API design and secure authentication patterns.



Backend in a Web Application

The backend is the powerhouse of any web application, running on a server and orchestrating all the behind-the-scenes operations that make modern web experiences possible.

It processes requests from the frontend, executes complex business logic, manages database operations, and returns formatted responses that the client can understand and display.

Popular backend languages include JavaScript (Node.js), Python, Java, Ruby, and PHP each offering unique strengths for different application needs.



Request Processing

Handles incoming client requests efficiently



Data Management

Securely accesses and manipulates databases



ARCHITECTURE

The Client–Server Model

Web applications operate on a fundamental architecture where clients and servers communicate through a standardized protocol.



Client

Browser or mobile app that initiates requests and displays results to users



HTTP Protocol

Communication layer that transfers data between client and server



Server

Backend application that processes requests and returns responses

HTTP Protocol Basics

Request-Response Protocol

HTTP follows a simple pattern: a client sends a request, and the server returns a response. This elegant design has powered the web for decades.

Stateless by Design

Each HTTP request is independent—the server doesn't remember previous interactions. This makes scaling easier but requires explicit state management through sessions or tokens.

Universal Communication

HTTP is the foundation of all web communication, enabling browsers, mobile apps, and servers to speak the same language regardless of platform.



STRUCTURE

Anatomy of an HTTP Request

Every HTTP request is a structured message containing multiple components that work together to communicate exactly what the client needs.

URL

The address of the resource you're trying to access

Method

The action you want to perform on that resource

Headers

Metadata about the request like content type and auth

Body

The actual data payload sent to the server

Query Params

Optional filters and parameters in the URL

HTTP Methods: Actions That Drive the Web

HTTP methods tell the server what action to perform on a resource. Each method has a specific purpose and expected behavior.

Method	Purpose	Example Use Case
GET	Read data	Fetch a list of users or a single profile
POST	Create new data	Submit a new user registration form
PUT	Replace data	Update all fields of a user profile
DELETE	Remove data	Delete a user account permanently

URLs, Query Parameters & Request Body

Different parts of an HTTP request carry information in different ways, each serving a specific purpose in the communication protocol.

1

URL Path Parameters

Identify specific resources directly in the URL structure

```
/users/5
```

This retrieves user with ID 5

2

Query Parameters

Filter or modify requests using key-value pairs after the ?

```
/users?role=admin&status=active
```

Returns only active admin users

3

Request Body (JSON)

Send complex data structures in the request payload

```
{  
  "email": "test@gmail.com",  
  "name": "John Doe",  
  "role": "developer"  
}
```

Commonly used with POST and PUT requests

Authentication vs Authorization

Authentication

Who are you?

Authentication verifies the identity of a user.
It's the process of confirming that someone is who they claim to be.

- Login with username and password
- Biometric verification (fingerprint, face ID)
- Two-factor authentication codes

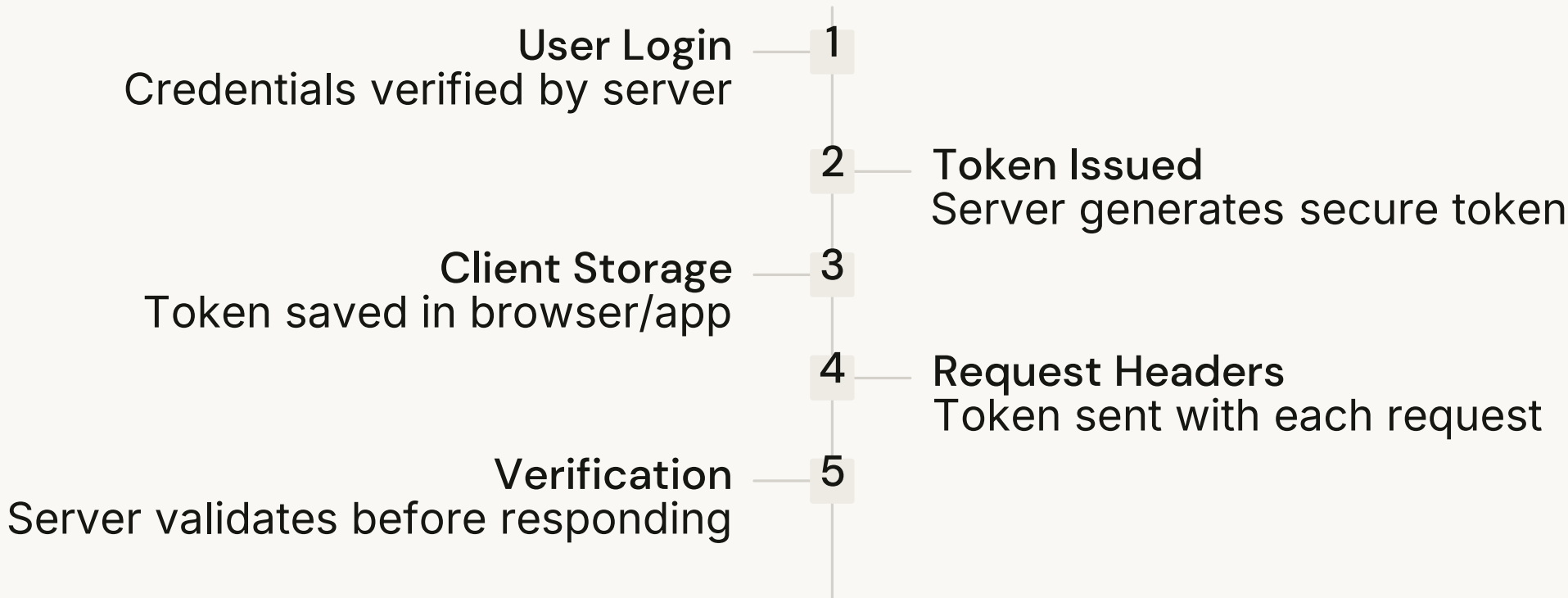
Authorization

What can you do?

Authorization determines what resources and actions an authenticated user is allowed to access.

- Admin-only dashboard access
- Read-only vs. edit permissions
- Role-based access control (RBAC)

Token-Based Authentication Flow



COMPETITION API DESCRIPTION

1

/api/register

POST

BODY: RollNumber

Returns

```
{
  RollNumber,
  message: "Registration successful",
  access_token: token
}
```

2

/api/confirm

POST

Headers: access_token

BODY: RollNumber

Returns

```
{
  RollNumber,
  message: "Registration successful",
  final_token: token
}
```

3

/api/finish

POST

Headers: access_token,
final_token

BODY: RollNumber

Returns

```
{
  RollNumber,
  message: "Registration successful",
  status: "completed",
  rank
}
```